

PARTE 1 — PLANO DE PROJETO

PP_2026_001_FORENSIA

Campo	Descrição
Nome do Projeto	ForensIA – Detecção Multimodal de Conteúdo Gerado por IA
Código	PP_2026_001_FORENSIA
Equipe	Rubens Rosa Faria Rodrigues, João Vitor Marques Moises Zancheta, Marcelo Rodrigues de Souza
Período	05/02/2026 – 05/11/2026
Versão	3.0

1. Visão Geral

1.1 Descrição

O ForensIA é um sistema de detecção de conteúdo gerado por IA que analisa texto, áudio, imagem e vídeo, retornando probabilidade (0–100%) e classificação (Humano, Híbrido ou IA). Desenvolvido como TCC por 3 estudantes em arquitetura de microservices.

1.2 Objetivo

Plataforma de detecção automatizada com acurácia $\geq 85\%$, API RESTful com JWT e conformidade com LGPD/GDPR/AI Act.

1.3 Justificativa

O avanço de LLMs (GPT, Gemini, Claude) torna conteúdos sintéticos indistinguíveis de produções humanas. O AI Act Europeu (Regulamento UE 2024/1689) e o PL 2338/2023 reforçam a urgência da temática.

1.4 Status

- (x) Em Andamento
- () Interrompido
- () Cancelado
- () Encerrado

1.5 Escopo

- Motor Python: FastAPI + HuggingFace + PyTorch (texto, áudio, imagem, vídeo).
- API REST Java: Spring Boot 3.3.5, JWT, Redis cache, PostgreSQL 15.
- Interface do usuário (Web ou Desktop — a definir). CI/CD: Docker + GitHub Actions.
- Fora do escopo: app mobile nativo, multilíngue além de PT/EN, certificação externa.

1.6 Riscos

ID	Risco	Categoria	I	P	IR
----	-------	-----------	---	---	----

R01	Baixa acurácia do modelo em produção	Técnico	4	3	12 — ALTO
R02	Mudanças nos LLMs invalidam modelo	Externo	4	4	16 — CRÍTICO
R03	Não conformidade com LGPD/GDPR	Legal	5	2	10 — MÉDIO
R04	Atraso na infraestrutura de nuvem	Organ.	3	3	9
R05	Saída de membro-chave da equipe	RH	4	2	8

1.7 Equipe

Papel	Nome	Responsabilidades
Líder / Backend Java	Rubens Rosa Faria Rodrigues	Coordenação, API REST, PostgreSQL, Docker, CI/CD
Motor de IA / Python	Marcelo Rodrigues de Souza	FastAPI, HuggingFace/PyTorch — texto, áudio, imagem e vídeo
Interface / Testes / Docs	João Vitor Marques Moises Zancheta	Frontend, JUnit 5, pytest, REST Assured, LGPD

1.8 Acompanhamento

Tipo	Periodicidade
Reunião de Status	Semanal
Relatório de Progresso	Quinzenal
Reunião de Marco	Ao atingir 50–55% de conclusão

1.9 Metodologia

PMBOK® 7ª Edição com desenvolvimento incremental por fases. ABNT NBR ISO/IEC 12207:2021 e IEEE Std 830. Versionamento via Git/GitHub com CI/CD por GitHub Actions.

PARTE 2 — REGRAS DE SISTEMA

RS_2026_001_FORENSIA

1. Parametrização

1.1 Análise de Texto (RF01–RF03)

ID	Funcionalidade	Obrigatoriedade	Prioridade
RF01–03	Submissão e Análise de Texto	Obrigatória	Alta

- Campo de texto obrigatório; análise não executada sem conteúdo.
- Idiomas: Português e Inglês (detecção automática).
- Retorna probabilidade com 2 casas decimais e classificação: Humano (0–30%), Híbrido (31–69%), IA (70–100%).
- Textos < 50 palavras: alerta de possível redução de acurácia.
- Tempo máximo (até 5.000 palavras): 2 segundos.
- LOG: data/hora, usuário, tamanho do texto, resultado.

1.2 Análise de Áudio (RF11)

- Formatos: MP3, WAV, AAC. Tamanho máximo: 50 MB. Duração máxima: 10 minutos.
- Taxa < 8 kHz → alerta de redução de acurácia.
- Classificação: Humano, IA Sintético, Híbrido. Arquivo não armazenado sem consentimento (LGPD).

1.3 Análise de Imagem (RF12)

- Formatos: PNG, JPG, WEBP. Tamanho máximo: 20 MB. Resolução mínima: 256×256 px.
- JPEG com qualidade < 60% → alerta. Classificação: Humano/Real, IA Gerada, Híbrida.

1.4 Análise de Vídeo (RF13)

- Formatos: MP4, AVI, MOV. Tamanho máximo: 200 MB. Duração máxima: 5 minutos.
- Análise da camada visual (frames) e de áudio com índices individuais e consolidado.
- Classificação: Autêntico, Deepfake/IA, Parcialmente Manipulado. Alerta se confiança < 70%.

1.5 API RESTful com JWT (RF04)

- Authorization: Bearer <token> obrigatório; sem token válido → HTTP 401.
- Tokens JWT com validade de 1 hora; renovação via refresh endpoint.
- Rate limiting: máx. 100 req/min por token → HTTP 429.
- Documentação automática via SpringDoc/OpenAPI 3.0 em /swagger-ui.html.

1.6 Histórico, Exportação e Painel Admin (RF07–RF09)

- Histórico: apenas para usuários autenticados; filtros por mídia, período e resultado.
- Armazena apenas metadados e resultado — não o conteúdo original (LGPD Art. 38).
- Exportação: PDF individual e CSV em lote; on-demand, não armazenados.

- Painel Admin: role ADMIN; CRUD de usuários, permissões e métricas (HTTP 403 para demais).

1.7 LOG

Ações registradas: autenticação, submissões, consultas, exportações, ações admin e erros. Conteúdo: data/hora UTC, ID usuário, ação, resultado, IP. Retenção mínima: 6 meses (Marco Civil da Internet – Lei 12.965/2014).

2. Validações

2.1 Formato e Tamanho

- Formato inválido → HTTP 400 com mensagem informativa.
- Tamanho excedido → HTTP 413 com limite informado.
- Arquivo corrompido → não iniciar processamento; solicitar novo envio.

2.2 Consentimento (LGPD)

- Cadastro exige aceite explícito da Política de Privacidade e Termos de Uso.
- Revogação: dados anonimizados/excluídos em até 72 horas.
- Modo anônimo: nenhum dado identificável armazenado.

2.3 Disponibilidade do Motor de IA

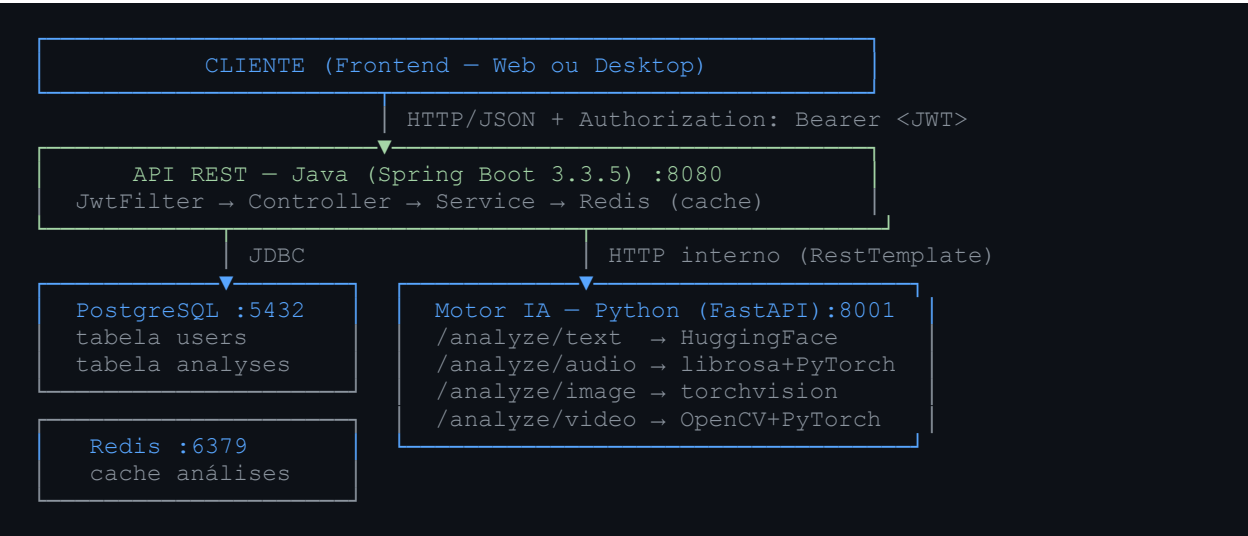
- Python indisponível → HTTP 503 + mensagem ao usuário.
- Tempo máximo de espera em fila: 30 segundos; após isso, requisição cancelada.

PARTE 3 — ARQUITETURA E STACK TECNOLÓGICA

Java 21 + Spring Boot 3.3.5 · Python 3.11 + FastAPI · PostgreSQL 15 · Redis 7 · Docker

1. Visão Arquitetural

Arquitetura de microservices: API Java autêntica JWT, consulta Redis e, em cache miss, delega ao motor Python. Resultados persistidos no PostgreSQL e cacheados no Redis.



2. Stack Tecnológica

Camada	Tecnologia	Justificativa
Backend / API	Java 21 + Spring Boot 3.3.5 + Maven	Ecosistema robusto; OpenAPI via SpringDoc
Motor de IA	Python 3.11 + FastAPI + HuggingFace + PyTorch + librosa + OpenCV	Líder em ML/NLP; modelos pré-treinados
Banco de Dados	PostgreSQL 15 + Redis 7	Relacional + cache de alto desempenho
Autenticação	JWT (JJWT 0.12.6) + BCrypt	Stateless; sem sessão
Documentação API	SpringDoc OpenAPI 2.5.0 (Swagger UI)	Documentação interativa automática
Infraestrutura	Docker + Docker Compose	Isolamento e portabilidade
CI/CD	GitHub Actions	Pipeline automatizado a cada push

3. Endpoints da API

Endpoint	Auth	Descrição
POST /auth/register	Pública	Cadastro de novo usuário
POST /auth/login	Pública	Login — retorna token JWT
POST /api/v1/analysis/text	JWT	Análise de texto livre
POST /api/v1/analysis/file?type=audio image video	JWT	Análise de arquivo multipart
GET /api/v1/history	JWT	Histórico de análises do usuário
DELETE /api/v1/history/{id}	JWT	Remove análise do histórico
GET /actuator/health	Pública	Health check Docker
GET /swagger-ui.html	Pública	Documentação Swagger UI

PARTE 4 — ESTRUTURA DE PASTAS E ARQUIVOS

Módulo Java (Spring Boot) + Módulo Python (FastAPI)

1. Visão Geral do Workspace

Workspace com dois módulos independentes orquestrados pelo docker-compose.yml na raiz.

```
forensia-workspace\
├── docker-compose.yml      ← orquestra PostgreSQL, Redis, Java, Python
├── .github\workflows\ci.yml ← pipeline CI/CD GitHub Actions
├── forensia\              ← módulo Java (Spring Boot)
└── forensia-ai-engine\    ← módulo Python (FastAPI)
```

2. Módulo Java — forensia/

```
forensia\
├── pom.xml                ← dependências Maven (Spring Boot 3.3.5)
├── Dockerfile             ← build multi-stage Maven → JRE 21
├── src\main\java\com\forensia\
│   ├── ForensiaApplication.java ← @SpringBootApplication + @EnableCaching
│   ├── config\
│   │   ├── SecurityConfig.java ← JWT filter chain, CORS, BCrypt bean
│   │   └── RestTemplateConfig.java ← cliente HTTP para o motor Python
│   ├── (timeouts)
│   │   └── OpenApiConfig.java ← Swagger UI + botão Authorize JWT
│   ├── controller\
│   │   ├── AuthController.java ← endpoints REST (@RestController)
│   │   ├── AnalysisController.java ← POST /auth/register, /auth/login
│   │   └── HistoryController.java ← POST /api/v1/analysis/text, /file
│   │                                     GET /api/v1/history, DELETE /{id}
│   ├── service\
│   │   ├── AuthService.java ← lógica de negócio (@Service)
│   │   └── UserDetailsService ← registro, login, implementa
│   │       ├── AnalysisService.java ← Redis → Python → PostgreSQL +
│   │       └── HistoryService.java ← consulta e exclusão de histórico
│   ├── repository\
│   │   ├── UserRepository.java ← acesso ao banco (@Repository)
│   │   └── AnalysisRepository.java ← JpaRepository<User, Long>
│   │                                     JpaRepository<Analysis, Long>
│   ├── model\
│   │   ├── User.java ← entidades JPA (@Entity)
│   │   └── Analysis.java ← tabela users + implements UserDetails
│   │                                     tabela analyses (histórico)
│   ├── dto\
│   │   ├── request\
│   │   │   ├── LoginRequest.java ← @NotBlank username + password
│   │   │   ├── RegisterRequest.java ← @Email, @Size(min=3,max=50), @Pattern
│   │   │   └── TextRequest.java ← @NotBlank @Size(min=10) text
│   │   └── response\
│   │       └── AnalysisResult.java ← probability, classification, mediaType,
│   │           warning, analyzedAt
│   │               ├── AuthResponse.java ← token, username, role, expiresIn
│   │               └── ApiResponse.java ← { success, data, error, timestamp }
│   ├── genérico
│   │   └── security\
│   │       └── JwtUtil.java ← gera / valida / extrai username do token
```

3. Módulo Python — forensia-ai-engine/

4. Fluxo de uma Análise

#	Etapa
1	Usuário → POST /auth/login {username,password} → JwtUtil.generateToken() → retorna token JWT
2	Usuário → POST /api/v1/analysis/text com Authorization: Bearer <token>

3	JwtFilter valida token → extrai username → autentica no SecurityContext
4	AnalysisController recebe TextRequest (@Valid) → delega para AnalysisService
5	AnalysisService consulta Redis: chave = text.hashCode()
6	Cache HIT → retorna AnalysisResult imediatamente (≈ 0 ms)
7	Cache MISS → RestTemplate.postForObject(aiEngineUrl+"/analyze/text", body, AnalysisResult.class)
8	Motor Python: HuggingFace pipeline processa texto → {probability, classification, warning}
9	Java salva Analysis no PostgreSQL via AnalysisRepository.save()
10	Java armazena no Redis com TTL de 1 hora para reutilização
11	Retorna HTTP 200 ApiResponse.ok(AnalysisResult) ao cliente

PARTE 5 — ESPECIFICAÇÃO DE REQUISITOS

SRS_2026_001_FORENSIA — IEEE Std 830 / ABNT NBR ISO/IEC 12207

1. Requisitos Funcionais (RF)

ID	Nome	Descrição	Prioridade	Status
RF01	Submissão de Texto	Aceita campo de texto livre para análise de conteúdo gerado por IA.	Alta	Desenvolvimento
RF02	Índice de Probabilidade	Retorna probabilidade 0–100% com 2 casas decimais.	Alta	Desenvolvimento
RF03	Classificação Multiclasse	Classifica em Humano (0–30%), Híbrido (31–69%) ou IA (70–100%).	Alta	Desenvolvimento
RF04	API RESTful com JWT	Expõe endpoints REST autenticados via JWT com documentação OpenAPI 3.0.	Alta	Desenvolvimento
RF05	Suporte PT/EN	Detecta automaticamente o idioma do texto submetido.	Alta	Em espera
RF06	Destaque de Trechos	Identifica segmentos com maior probabilidade de geração por IA.	Média	Em espera
RF07	Histórico de Análises	Mantém histórico filtrável por tipo, período e resultado.	Média	Em espera
RF08	Exportação	Exporta resultados em PDF (individual) e CSV (lote, UTF-8).	Baixa	Em espera
RF09	Painel Administrativo	Painel ADMIN para gestão de usuários, permissões e métricas.	Média	Em espera
RF10	Processamento em Lote	Aceita múltiplas análises em uma única requisição via API.	Baixa	Em espera
RF11	Análise de Áudio	Aceita MP3/WAV/AAC (máx. 50 MB/10 min); detecta síntese por IA.	Alta	Desenvolvimento
RF12	Análise de Imagem	Aceita PNG/JPG/WEBP (máx. 20 MB); detecta geração por IA.	Alta	Desenvolvimento
RF13	Análise de Vídeo	Aceita MP4/AVI/MOV (máx. 200 MB/5 min); analisa frames e áudio.	Alta	Desenvolvimento

2. Requisitos Não Funcionais (RNF)

ID	Categoria	Descrição	Critério de Aceite
RNF01	Desempenho	Latência API ≤ 2 s para textos de até 5.000 palavras.	Testes de carga com JMeter

RNF02	Segurança	HTTPS/TLS 1.3. Senhas com BCrypt (custo 10). Sem dados sensíveis em LOG.	Pentest + code review
RNF03	Privacidade	Arquivos/textos não persistidos além do processamento sem consentimento.	Auditoria LGPD / DPIA
RNF04	Escalabilidade	Suporte a N requisições simultâneas sem degradação acima do SLA.	Testes de estresse (JMeter)
RNF05	Disponibilidade	SLA de 99,5% mensais em produção.	Monitoramento contínuo
RNF06	Acessibilidade	Interface web em conformidade com WCAG 2.1 nível AA.	Auditoria de acessibilidade
RNF07	Manutenibilidade	Cobertura de testes $\geq 80\%$ (JUnit 5 + pytest).	JaCoCo + pytest-cov
RNF08	Portabilidade	Sistema containerizado via Docker; múltiplos provedores de nuvem.	Testes em ambientes distintos
RNF09	Acurácia do Modelo	Acurácia $\geq 85\%$; F1-Score $\geq 0,82$; falsos positivos $\leq 10\%$.	Dataset de teste balanceado
RNF10	Rastreabilidade	LOG de todas as operações retido por no mínimo 6 meses.	Auditoria de logs
RNF11	Interoperabilidade	API OpenAPI 3.0; respostas padronizadas ApiResponse<T>.	REST Assured
RNF12	Resiliência	Circuit breaker: HTTP 503 em caso de indisponibilidade ≤ 30 s.	Testes de falha de serviço

3. Requisitos de Interface

3.1 Interface do Usuário

- Campo de texto com contador de palavras em tempo real.
- Upload de arquivos com validação de formato e tamanho antes do envio.
- Barra de progresso para análises de áudio, imagem e vídeo.
- Gauge (0–100%) com badge de classificação no resultado.
- Histórico paginado com filtros e botões de exportação PDF/CSV.

3.2 Interface de Comunicação (API)

- Authorization: Bearer <token> em todas as requisições autenticadas.
- Respostas padronizadas: { success, data, error, timestamp } — ApiResponse<T>.
- Upload via multipart/form-data. Análise de texto via application/json.
- HTTP 200/201/400/401/403/409/413/429/503 conforme o cenário.

4. Rastreabilidade RF × RS × UC

RF	Nome	Regras de Sistema	Caso de Uso
----	------	-------------------	-------------

RF01	Submissão de Texto	RS 1.1.1, RS 2.1	UC03 — Analisar Texto
RF03	Classificação	RS 1.1.1–1.1.4	UC03, UC04, UC05, UC06
RF04	API REST com JWT	RS 1.1.5	UC01, UC02
RF07	Histórico	RS 1.1.6 (LGPD Art. 38)	UC07 — Consultar Histórico
RF08	Exportação	RS 1.1.7	UC09 — Exportar Resultados
RF09	Painel Admin	RS 1.1.8	UC10 — Administrar Usuários
RF11	Análise de Áudio	RS 1.1.2 (MP3/WAV/AAC, 50 MB)	UC04 — Analisar Áudio
RF12	Análise de Imagem	RS 1.1.3 (PNG/JPG/WEBP, 20 MB)	UC05 — Analisar Imagem
RF13	Análise de Vídeo	RS 1.1.4 (MP4/AVI/MOV, 200 MB)	UC06 — Analisar Vídeo

5. Métricas de Qualidade do Modelo

Métrica	Meta Mínima	Modo de Avaliação	Observação
Acurácia Global	≥ 85%	Dataset de teste balanceado	Por tipo de mídia
Precisão (Precision)	≥ 80%	pytest + HuggingFace Evaluate	Classe "gerado por IA"
Recall	≥ 80%	pytest + HuggingFace Evaluate	Classe "gerado por IA"
F1-Score	≥ 0,82	Média harmônica P × R	Balanceia FP/FN
Falsos Positivos	≤ 10%	Dataset de textos humanos	Textos humanos → IA
Latência API	≤ 2 s	JMeter — 5.000 palavras	Cache miss
Disponibilidade	≥ 99,5%/mês	Monitoramento contínuo	Em produção

PARTE 6 — CASOS DE USO

UC_2026_001_FORENSIA

1. Atores

Ator	Tipo	Descrição
Usuário Comum	Primário	Realiza análises, consulta histórico e exporta resultados.
Administrador	Primário	Gerencia usuários, permissões e monitora métricas.
API Java (Spring Boot)	Secundário	Orquestra autenticação, cache, chamadas ao motor Python e persistência.
Motor IA (Python/FastAPI)	Secundário	Executa modelos HuggingFace/PyTorch e retorna probabilidade.
PostgreSQL	Secundário	Persiste usuários e histórico de análises.

2. Especificação dos Casos de Uso

UC01 — Registrar-se	
Atores	Usuário Comum (primário)
Pré-condições	Usuário não possui conta no sistema.
Fluxo Principal	1. POST /auth/register com {username, email, password}. 2. @Valid valida campos (NotBlank, Size, Email, Pattern). 3. Verifica unicidade: existsByUsername() e existsByEmail(). 4. BCrypt.encode(password). 5. userRepository.save(user). 6. HTTP 201 com ApiResponse.ok(null).
Fluxo Alternativo	3a. Username ou e-mail duplicado → HTTP 409 "Username ou e-mail já está em uso".
Pós-condições	Conta criada. Usuário pode autenticar-se.

UC02 — Autenticar-se (Login)	
Atores	Usuário Comum (primário)
Pré-condições	Conta cadastrada no sistema.
Fluxo Principal	1. POST /auth/login com {username, password}. 2. findByUsername() → orElseThrow. 3. BCrypt.matches(senha, hash). 4. JwtUtil.generateToken(username). 5. HTTP 200 com AuthResponse {token, username, role, expiresIn}.
Fluxo Alternativo	2a/3a. Credenciais inválidas → HTTP 500 "Credenciais inválidas" (sem revelar qual campo).

Pós-condições	Cliente recebe token JWT para incluir em requisições subsequentes.
----------------------	--

UC03 — Analisar Texto

Atores	Usuário Comum (primário), API Java, Motor IA Python
Pré-condições	Usuário autenticado com token JWT válido.
Fluxo Principal	1. POST /api/v1/analysis/text {text} + Bearer <token>. 2. JwtFilter valida token. 3. @Valid valida TextRequest. 4. AnalysisService verifica Redis (chave=text.hashCode()). 5. Cache MISS → RestTemplate chama /analyze/text. 6. Python: HuggingFace pipeline → {probability, classification, warning}. 7. Save Analysis + Redis TTL 1h. 8. HTTP 200 AnalysisResult.
Fluxo Alternativo	4. Cache HIT → retorna sem chamar Python. 5a. Python offline → HTTP 503.
Pós-condições	Resultado disponível; análise persistida no histórico.

UC04 — Analisar Áudio

Atores	Usuário Comum (primário), API Java, Motor IA Python
Pré-condições	Usuário autenticado com token JWT válido.
Fluxo Principal	1. POST /api/v1/analysis/file?type=audio com multipart file. 2. Valida formato (MP3/WAV/AAC) e tamanho (≤ 50 MB). 3. Monta ByteArrayResource. 4. RestTemplate → /analyze/audio. 5. Python: librosa MFCC → PyTorch classifier → {probability, classification, warning}. 6. Save + HTTP 200.
Fluxo Alternativo	2a. Formato inválido → HTTP 400. 2b. Arquivo > 50 MB → HTTP 413. 4a. Python offline → HTTP 503.
Pós-condições	Análise de áudio registrada com mediaType="AUDIO".

UC05 — Analisar Imagem

Atores	Usuário Comum (primário), API Java, Motor IA Python
Pré-condições	Usuário autenticado com token JWT válido.
Fluxo Principal	1. POST /api/v1/analysis/file?type=image com multipart file. 2. Valida formato (PNG/JPG/WEBP) e tamanho (≤ 20 MB). 3. RestTemplate → /analyze/image. 4. Python: PIL + torchvision transforms → ResNet-50 → {probability, classification}. 5. Save + HTTP 200.
Fluxo Alternativo	2a. Formato inválido → HTTP 400. 2b. > 20 MB → HTTP 413. 4a. Resolução $< 256 \times 256$ → warning.
Pós-condições	Análise de imagem registrada com mediaType="IMAGE".

UC06 — Analisar Vídeo

Atores	Usuário Comum (primário), API Java, Motor IA Python
Pré-condições	Usuário autenticado com token JWT válido.
Fluxo Principal	1. POST /api/v1/analysis/file?type=video com multipart file. 2. Valida formato (MP4/AVI/MOV), tamanho (≤ 200 MB) e duração (≤ 5 min). 3. RestTemplate →

	/analyze/video. 4. Python: OpenCV frames + librosa áudio → PyTorch → índices individual e consolidado. 5. Save + HTTP 200.
Fluxo Alternativo	2a. Inválido → HTTP 400/413. 4a. Confiança < 70% → campo warning preenchido.
Pós-condições	Análise de vídeo registrada com mediaType="VIDEO".

UC07 — Consultar Histórico

Atores	Usuário Comum (primário)
Pré-condições	Usuário autenticado.
Fluxo Principal	1. GET /api/v1/history + Bearer <token>. 2. @AuthenticationPrincipal extrai username. 3. HistoryService.getHistory() → findByUsernameOrderByCreatedAtDesc(). 4. HTTP 200 List<AnalysisResult>.
Pós-condições	Usuário visualiza histórico ordenado por data decrescente.

UC08 — Excluir Análise

Atores	Usuário Comum (primário)
Pré-condições	Usuário autenticado; análise pertence ao usuário.
Fluxo Principal	1. DELETE /api/v1/history/{id} + Bearer <token>. 2. findById(id) → orElseThrow. 3. Verifica ownership (analysis.getUsername().equals(username)). 4. delete(analysis). 5. HTTP 200 ApiResponse.ok(null).
Fluxo Alternativo	2a. ID não encontrado → RuntimeException → HTTP 500. 3a. Ownership falhou → "Sem permissão".
Pós-condições	Análise removida permanentemente.

UC09 — Exportar Resultados

Atores	Usuário Comum (primário)
Pré-condições	Usuário autenticado com histórico disponível.
Fluxo Principal	1. Usuário solicita exportação PDF (individual) ou CSV (lote). 2. Sistema gera on-demand do histórico filtrado. 3. Arquivo entregue ao cliente; nenhum armazenamento no servidor. 4. LOG registrado.
Pós-condições	Arquivo disponibilizado para download.

UC10 — Administrar Usuários

Atores	Administrador (primário)
Pré-condições	Usuário autenticado com role ADMIN.
Fluxo Principal	1. Acessa painel via token ADMIN. 2. Visualiza métricas: total de análises por tipo, período e usuário. 3. CRUD de contas de usuário. 4. Atribui/revoga permissões por funcionalidade. 5. Todas as ações registradas em LOG de auditoria.
Fluxo Alternativo	1a. Role != ADMIN → SecurityConfig retorna HTTP 403 Forbidden.

Pós-condições	Alterações aplicadas; LOG de auditoria atualizado.
----------------------	--